

Chap2

- 등차 수열
- 피보나치 (재귀 버전)
- 피보나치 (비재귀 버전)
- 하노이탑 (재귀 버전)
- 선택정렬 (재귀 버전)
- 전위/중위/후위 표현법
- 깊이 우선 탐색

1. 재귀 알고리즘

- 1. 경계조건 (종료 조건) : 재귀 호출이 반복되다 궁극적으로 끝나는 조건.
- 2. 재귀 호출
- 3. 관계 : 닮은 꼴 작은 문제와 본 문제간의 관계를 나타내는 부분

2. 등차 수열

```
In [7]: def seq(n) :  
        if(n==1) :  
            return 1  
        else :  
            return seq(n-1)+n  
  
        print(seq(3))  
        print(seq(4))  
        print(seq(5))
```

```
6  
10  
15
```

3. 피보나치

```
In [17]: #피보나치 수열 (재귀 버전)
def fibonacci(n) :
    if(n==1 or n==2) :
        return 1
    else :
        return fibonacci(n-1)+fibonacci(n-2)

print(fibonacci(1))
print(fibonacci(2))
print(fibonacci(3))
print(fibonacci(4))
```

```
1
1
2
3
```

```
In [16]: #피보나치 수열 (비재귀 버전)

def fibonacci( n ) :
    arr = [1,1]
    if(n==1 or n==2) :
        return 1

    for i in range(2,n) :
        arr.append(arr[i-1]+arr[i-2])
    return arr[n-1]

print( fibonacci(1))
print( fibonacci(2))
print( fibonacci(3))
print( fibonacci(4))
```

```
1
1
2
3
```

4.하노이탑

- 기둥 a에 있는 n개의 원반을 기둥 C를 보조 기둥으로 사용해 기둥 b로 옮긴다.
- move(n,a,b,c) :
- 1). if(n>0) :
- 2). move(n-1,a,c,b)
- 3). a에 있는 원반을 b로 옮긴다.
- 4). move(n-1,c,b,a)
- 기둥 a에 n개의 원반을 기둥 b로 옮기려면, 기둥 c에 n-1개의 원반을 우선 옮겨야한다.
- 그 다음, 기둥 a에 남은 원반을 기둥 c로 옮기면 된다.
- 원반이 하나이면 그냥 옮기면 된다.

```
In [26]: def hanoi ( n, start,end, via) :      # n : 원판의 개수, start : 출발 기
동, end : 도착 기둥, via : 경유 기둥

    if(n==1) :
        print(f"{start}에서 {end}로 원판 1개 이동")
    else :
        hanoi(n-1,start,via,end)
        hanoi(n-1,via,end,start)

    print(" # A에서 C로 원판 3개 이동, B는 경유지")
    hanoi(3,'A','C','B') # A에서 C로 원판 3개 이동, B는 경유지

    print(" # A에서 C로 원판 4개 이동, B는 경유지")
    hanoi(4,'A','C','B') # A에서 C로 원판 4개 이동, B는 경유지
```

```
    # A에서 C로 원판 3개 이동, B는 경유지
A에서 C로 원판 1개 이동
C에서 B로 원판 1개 이동
B에서 A로 원판 1개 이동
A에서 C로 원판 1개 이동
    # A에서 C로 원판 4개 이동, B는 경유지
A에서 B로 원판 1개 이동
B에서 C로 원판 1개 이동
C에서 A로 원판 1개 이동
A에서 B로 원판 1개 이동
B에서 C로 원판 1개 이동
C에서 A로 원판 1개 이동
A에서 B로 원판 1개 이동
B에서 C로 원판 1개 이동
```

3.선택 정렬

- 가장 작은 값을 하나씩 골라서 앞쪽부터 정렬하는 방식
-
- 동작원리
- *1.전체 배열에서 최솟값 찾기
- *2.그 값을 맨 앞과 교환
- *3.그 다음 위치부터 다시 반복

```
In [30]: def selection_sort(arr):
          n = len(arr)
          print("Original array:", arr)
          for i in range(n):
              min_idx = i

              for j in range(i + 1, n):
                  if arr[j] < arr[min_idx]:
                      min_idx = j

              arr[i], arr[min_idx] = arr[min_idx], arr[i]
              print(arr) # 각 단계에서 배열의 상태 출력
          return arr

arr = [64, 25, 12, 22, 11]
sorted_arr = selection_sort(arr)
print("Sorted array:", sorted_arr)
```

```
Original array: [64, 25, 12, 22, 11]
[11, 25, 12, 22, 64]
[11, 12, 25, 22, 64]
[11, 12, 22, 25, 64]
[11, 12, 22, 25, 64]
[11, 12, 22, 25, 64]
Sorted array: [11, 12, 22, 25, 64]
```

4.전위, 중위, 후위 표현법

- 전위 (연산자가 앞에 옴)
- 중위 (연산자가 중간에 옴)
- 후위 (연산자가 뒤에 옴)
-

- 수식: $(A + B) * C$
- 전위 (Prefix): $* + A B C$
- 중위 (Infix): $(A + B) * C$
- 후위 (Postfix): $A B + C *$

Chap3 알고리즘 성능 _ 시간 복잡도 보는 법.

- Big-O (빅오) , $O(n)$: 최악의 경우(하한)
- $f(n) \leq c \times g(n)$ 일 때, $f(n) = O(g(n))$
- Big-Omega (빅오메가) , $\Omega(n)$: 최선의 경우
- $f(n) \geq c \times g(n)$ 일 때, $f(n) = \Omega(g(n))$
- Big-Theta (빅세타) , $\Theta(n)$: 최소와 최악의 중간인 평균적인 복잡도
- $c_1 \times f(n) \leq c_2 \times g(n)$ 일 때, $f(n) = \Theta(g(n))$

Chap4 파이썬 기초

1.파이썬 언어 특징

1). 컴파일러 언어 (C, C++ 해당 0)

- 먼저 코드를 번역하여 기계어로 한번에 바꾼다.
- 빠르다.
- 컴파일 시점에 에러난다.
- 테스트 느리다.

2). 인터프리터 언어 (파이썬 해당 0)

- 한 줄씩 번역하여 실행한다.
- 느리다.
- 인터프리터 시점에 에러난다. (런타임 에러)
- 테스트 빠르다.

3).용통성이 좋다 : 변수타입이 동적이다.

4).리스트를 다루는 기능이 좋다.

5).들여쓰기로 주종관계 표현한다.

2.파이썬 문법 기초

- 출력
- 문자열
- if/while/for/switch
- 불변/가변 (immutable/mutable)
- 리스트, 튜플, 딕셔너리, 집합
- 복사호출, 참조호출, 할당 호출
- 클래스
- 순회가능 클래스

In [2]:

```
# 출력
print("Hello")
x=77
print("x is", x,"th")

# 입력
print("5's element is", end = "")
print("Test")

# 중간문자
print('My', 'Name','Is','Tom', sep = '-')

```

```
Hello
x is 77 th
5's element isTest
My-Name-Is-Tom

```

```
In [4]: # % 나머지 연산
# // 몫 연산
# ** 지수 연산

# 문자열 연산
list = ['lion', 'tiger', 'bear']
print(list)
print("Hello")
print('5"s element')
print('''5"s element ''')

# escape 문자
print("4\s element \\ 6\s element")
```

```
['lion', 'tiger', 'bear']
Hello
5"s element
5's element
4's element \ 6"s element
```

```
In [6]: # for문
sum = 0
for idx in range(1, 11) :
    sum += idx

print("1부터 10까지의 합은", sum)
```

```
1부터 10까지의 합은 55
```

```
In [8]: # while 문
sum = 0; i = 3
while i <= 10 :
    sum += i
    i += 1

print("3부터 10까지의 합은", sum)
```

```
3부터 10까지의 합은 52
```

In [9]:

```
# if문

i =5
if i<10 :
    print("i는 10보다 작습니다.")
else :
    print("i는 10보다 크거나 같습니다.")

#
if i<10 :
    print("i는 10보다 작습니다.")
elif i<20 :
    print("i는 10보다 크거나 같고, 20보다 작습니다.")
else :
    print("i는 20보다 크거나 같습니다.")
```

i는 10보다 작습니다.

In [21]:

```
# for 변수 in 순회 가능한 집합 :
#   수행할 문장
for i in range(10) :
    print(i, end = " ")
print('\n')

for i in range(4,10,2):
    print(i, end = " ")
print('\n')

for i in range(10,4,-2):
    print(i, end = " ")
print('\n')
```

0 1 2 3 4 5 6 7 8 9

4 6 8

10 8 6


```
In [23]: for i in [0,1,2,3]:
          print(i, end = " ")
          print('\n')

          s= [0,1,2,3]
          for i in s :
              print(i, end = " ")
              print('\n')

          for i in (0,1,2,3) :
              print(i, end = " ")
```

0 1 2 3

0 1 2 3

0 1 2 3

```
In [27]: # 딕셔너리
          d = {'a':1, 'b':2, 'c':3}
          for key in d :
              print(key,':', d[key], end = " ")
              print('\n')

          # 집합
          for i in {0,1,2,3} :
              print(i, end = " ")
              print('\n')

          #문자열
          for i in 'testShit' :
              print(i, end = "-")
              print('\n')
```

a : 1 b : 2 c : 3

0 1 2 3

t-e-s-t-S-h-i-t-

In [28]:

```
# <불변 타입>
# 값을 직접 변경할 수 없음.
# 정수 (int)
a = 10
print(id(a))

a = a + 5 # 새로운 객체 생성
print(a)
print(id(a)) # id가 바뀜

# 문자열 (str)
s = "hello"
print(id(s))

s += " world" # 새로운 문자열 생성
print(s)
print(id(s)) # id 변경됨

# 튜플 (tuple)
t = (1, 2, 3)
# t[0] = 10 # ❌ 오류 발생 (수정 불가)
```

```
140719331325128
15
140719331325288
2372848080848
hello world
2372864890928
```

In [29]:

```
# <가변 타입> (mutable type)
# 값을 직접 변경할 수 있음.

# 리스트 (list)
lst = [1, 2, 3]
print(id(lst))

lst.append(4) # 기존 객체 수정
print(lst)
print(id(lst)) # id 그대로 유지

# 딕셔너리 (dict)
d = {"a": 1, "b": 2}
d["c"] = 3 # 값 추가
print(d)

# 집합 (set)
s = {1, 2, 3}
s.add(4)
print(s)
```

```
2372852070400
[1, 2, 3, 4]
2372852070400
{'a': 1, 'b': 2, 'c': 3}
{1, 2, 3, 4}
```

In [32]:

```

# 리스트
a = [1, 3, 5, 7]
print(a)

for x in a:
    print(x, end = ':')
print('\n')

#튜플
t = (1, 3, 5, 7)
print(t)

for x in t:
    print(x, end = ':')
print('\n')

# 딕셔너리
d = {1:10, 3:20, 5:30, 10:40}
d[3] = 222
d[50] = 55

for i in d:
    print(i, ': ', d[i], end = ',')
print('\n')

#집합
a = {1, 3, 5, 7}
a.add(3) # 아무 영향 없음 (중복 불허)
a.add(10) # 집합 a에 10을 추가
print(a)

```

[1, 3, 5, 7]

1:3:5:7:

(1, 3, 5, 7)

1:3:5:7:

1 : 10, 3 : 222, 5 : 30, 10 : 40, 50 : 55,

{1, 3, 5, 7, 10}

복사호출, 참조호출, 할당호출

- 복사호출(Call by Value)
- 참조호출(Call by Reference)

[1] 복사호출 (Call by Value)

함수에 값을 전달할 때 "값의 복사본"이 전달되는 방식
원본 변수와 함수 내부 변수는 서로 다른 메모리를 사용
함수 내부에서 값을 변경해도 원본에는 영향 없음

```
In [33]: #[1] 복사호출 (Call by Value)

#   함수에 값을 전달할 때 "값의 복사본"이 전달되는 방식
#   원본 변수와 함수 내부 변수는 서로 다른 메모리를 사용
#   함수 내부에서 값을 변경해도 원본에는 영향 없음

def func(x):
    x = x + 10

a = 5
func(a)
print(a) # 5 (변화 없음)
```

5

```
In [ ]: #[2] 참조호출 (Call by Reference)

#   함수에 "변수의 주소(참조)"가 전달되는 방식
#   함수 내부에서 값을 변경하면 원본에도 영향이 있음
#   같은 객체를 공유한다고 보면 됨

def func(lst):
    lst.append(4)

a = [1, 2, 3]
func(a)
print(a) # [1, 2, 3, 4] (원본 변경됨)
```

In []:

```

#[3] 할당 호출 (Call by Assignment) ← 파이썬 방식

# 파이썬은 복사호출도, 참조호출도 아닌 "객체 참조 전달" 방식
# 변수는 값을 담는 것이 아니라 "객체를 가리킴"
# 함수에 전달될 때 객체의 참조가 복사됨

# 핵심:
# 불변 객체 → 값 변경 시 새 객체 생성 (복사호출처럼 동작)
# 가변 객체 → 내부 값 변경 가능 (참조호출처럼 동작)

#예시 1 (불변 객체):
def func(x):
    x = x + 10

a = 5
func(a)
print(a) # 5

#예시 2 (가변 객체):
def func(lst):
    lst.append(4)

a = [1, 2, 3]
func(a)
print(a) # [1, 2, 3, 4]

#[핵심 한 줄 정리]

# 복사호출 → 값만 복사 (원본 영향 없음)
# 참조호출 → 주소 공유 (원본 영향 있음)
# 파이썬 → 객체 참조 복사 (상황에 따라 둘 다처럼 보임)

```

In []:

클래스

```
In [ ]: class Sample :
        def __init__(self): # 생성자
            self.counter =0

        def isEmpty(self):
            return False

        def size(self):
            return False

        def insert(self,item):
            return False

        def numEven(n):
            for k in range(n):
                if k%2==0:
                    self.counter +=1
```

패키지와 모듈

- from 모듈이름 import* # 해당 모듈에 있는 모든 것을 가져온다.
- from 모듈이름 import 클래스이름 # 해당 모듈에서 특정 클래스만 가져온다.

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).